

Лабораторная работа №5

Стек, очередь, дек, приоритетная очередь

Цель работы:

1. Реализовать основные линейные структуры данных: стек, очередь, дек и приоритетную очередь
2. Освоить принципы LIFO (стек) и FIFO (очередь)
3. Научиться реализовывать эти структуры на основе динамического массива и связного списка
4. Изучить применение стека для решения задач (проверка скобок, обратная польская запись)
5. Реализовать приоритетную очередь на основе кучи (heap)

Теоретическое обоснование

1. Стек (Stack)

Стек - это структура данных, работающая по принципу LIFO (Last In - First Out, «последним пришёл - первым ушел»).

Аналогия: Стопка тарелок - последнюю поставленную тарелку мы возьмём первой.

Основные операции:

Операция	Описание	Сложность
push(value)	Добавляет элемент на вершину стека	O(1)
pop()	Удаляет и возвращает элемент с вершины	O(1)
top()	Возвращает элемент с вершины без удаления	O(1)
isEmpty()	Проверяет, пуст ли стек	O(1)
size()	Возвращает количество элементов	O(1)

Визуализация стека:

```
push(1):  [1]
push(2):  [1, 2]
push(3):  [1, 2, 3]
pop():    [1, 2]   (возвращает 3)
top():    [1, 2]   (возвращает 2, не удаляя)
```

2. Очередь (Queue)

Очередь - это структура данных, работающая по принципу FIFO (First In - First Out, «первым пришёл - первым ушел»).

Аналогия: Обычная очередь в магазине - первый в очереди обслуживается первым.

Основные операции:

Операция	Описание	Сложность
enqueue(value)	Добавляет элемент в конец очереди	O(1)
dequeue()	Удаляет и возвращает элемент из начала	O(1)
front()	Возвращает элемент из начала без удаления	O(1)
isEmpty()	Проверяет, пуста ли очередь	O(1)
size()	Возвращает количество элементов	O(1)

Визуализация очереди:

enqueue(1): [1]

enqueue(2): [1, 2]

enqueue(3): [1, 2, 3]

dequeue(): [2, 3] (возвращает 1)

front(): [2, 3] (возвращает 2, не удаляя)

3. Дек (Deque - Double-Ended Queue)

Дек - это структура данных, позволяющая добавлять и удалять элементы с обоих концов.

Основные операции:

Операция	Описание	Сложность
push_front(value)	Добавляет элемент в начало	O(1)
push_back(value)	Добавляет элемент в конец	O(1)
pop_front()	Удаляет элемент из начала	O(1)

Операция	Описание	Сложность
pop_back()	Удаляет элемент из конца	O(1)
front()	Возвращает первый элемент	O(1)
back()	Возвращает последний элемент	O(1)

4. Приоритетная очередь (Priority Queue)

Приоритетная очередь - это структура данных, в которой каждый элемент имеет приоритет. Элемент с наибольшим приоритетом извлекается первым.

Основные операции:

Операция	Описание	Сложность
push(value)	Добавляет элемент	O(log n)
pop()	Удаляет элемент с наибольшим приоритетом	O(log n)
top()	Возвращает элемент с наибольшим приоритетом	O(1)

Реализация: Обычно на основе двоичной кучи (heap).

Примеры выполнения практической части

Пример 1. Стек

Задача: Реализовать класс стека и проверить правильность расстановки скобок.

```
#include <iostream>
#include <string>
#include <stdexcept>

template <typename T>
class Stack {
private:
    T* data;
    int topIndex;
    int capacity;
```

```

void resize() {
    int newCapacity = capacity * 2;
    T* newData = new T[newCapacity];
    for (int i = 0; i <= topIndex; i++) {
        newData[i] = data[i];
    }
    delete[] data;
    data = newData;
    capacity = newCapacity;
}

```

public:

```

Stack(int initialCapacity = 10) : capacity(initialCapacity), topIndex(-1) {
    data = new T[capacity];
}

```

```

~Stack() { delete[] data; }

```

```

void push(const T& value) {
    if (topIndex >= capacity - 1) resize();
    data[++topIndex] = value;
}

```

```

T pop() {
    if (isEmpty()) throw std::underflow_error("Stack is empty");
    return data[topIndex--];
}

```

```

T top() const {
    if (isEmpty()) throw std::underflow_error("Stack is empty");
    return data[topIndex];
}

```

```

bool isEmpty() const { return topIndex == -1; }
int size() const { return topIndex + 1; }

```

```
};
```

```

bool checkBrackets(const std::string& expr) {
    Stack<char> stack;
    for (char ch : expr) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push(ch);

```

```

    } else if (ch == ')' || ch == ']' || ch == '}') {
        if (stack.isEmpty()) return false;
        char top = stack.pop();
        if ((ch == ')' && top != '(') ||
            (ch == ']' && top != '[') ||
            (ch == '}' && top != '{')) {
            return false;
        }
    }
}

return stack.isEmpty();
}

int main() {
    std::cout << "() : " << (checkBrackets("(") ? "BEPHO" : "HEBEPHO")
<< std::endl;
    std::cout << "([]) : " << (checkBrackets("([])") ? "BEPHO" :
"HEBEPHO") << std::endl;
    std::cout << "((())) : " << (checkBrackets("((()))") ? "BEPHO" :
"HEBEPHO") << std::endl;
    return 0;
}

```

Пример 2. Очередь

Задача: Реализовать класс очереди на кольцевом буфере.

```

#include <iostream>
#include <stdexcept>

```

```

template <typename T>
class Queue {
private:
    T* data;
    int frontIndex;
    int rearIndex;
    int count;
    int capacity;

    void resize() {
        int newCapacity = capacity * 2;
        T* newData = new T[newCapacity];
        for (int i = 0; i < count; i++) {
            newData[i] = data[(frontIndex + i) % capacity];
        }
    }
};

```

```

    }
    delete[] data;
    data = newData;
    frontIndex = 0;
    rearIndex = count;
    capacity = newCapacity;
}

```

public:

```

Queue(int initialCapacity = 10)
    : capacity(initialCapacity), frontIndex(0), rearIndex(0), count(0) {
    data = new T[capacity];
}

```

```

~Queue() { delete[] data; }

```

```

void enqueue(const T& value) {
    if (count >= capacity) resize();
    data[rearIndex] = value;
    rearIndex = (rearIndex + 1) % capacity;
    count++;
}

```

```

T dequeue() {
    if (isEmpty()) throw std::underflow_error("Queue is empty");
    T value = data[frontIndex];
    frontIndex = (frontIndex + 1) % capacity;
    count--;
    return value;
}

```

```

T front() const {
    if (isEmpty()) throw std::underflow_error("Queue is empty");
    return data[frontIndex];
}

```

```

bool isEmpty() const { return count == 0; }
int size() const { return count; }
};

```

```

int main() {
    Queue<int> q;
}

```

```

q.enqueue(10);
q.enqueue(20);
q.enqueue(30);
std::cout << "dequeue: " << q.dequeue() << std::endl;
std::cout << "front: " << q.front() << std::endl;
return 0;
}

```

Пример 3. Дек

Задача: Реализовать класс дека и проверить, является ли строка палиндромом.

```

#include <iostream>
#include <string>
#include <stdexcept>

template <typename T>
class Deque {
private:
    struct Node {
        T data;
        Node* prev;
        Node* next;
        Node(const T& value) : data(value), prev(nullptr), next(nullptr) {}
    };

    Node* head;
    Node* tail;
    int count;

public:
    Deque() : head(nullptr), tail(nullptr), count(0) {}

    ~Deque() {
        while (head != nullptr) {
            Node* temp = head;
            head = head->next;
            delete temp;
        }
    }

    void push_front(const T& value) {

```

```

Node* newNode = new Node(value);
if (head == nullptr) {
    head = tail = newNode;
} else {
    newNode->next = head;
    head->prev = newNode;
    head = newNode;
}
count++;
}

void push_back(const T& value) {
    Node* newNode = new Node(value);
    if (tail == nullptr) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    count++;
}

T pop_front() {
    if (isEmpty()) throw std::underflow_error("Deque is empty");
    Node* temp = head;
    T value = temp->data;
    head = head->next;
    if (head != nullptr) head->prev = nullptr;
    else tail = nullptr;
    delete temp;
    count--;
    return value;
}

T pop_back() {
    if (isEmpty()) throw std::underflow_error("Deque is empty");
    Node* temp = tail;
    T value = temp->data;
    tail = tail->prev;
    if (tail != nullptr) tail->next = nullptr;
    else head = nullptr;
}

```

```

        delete temp;
        count--;
        return value;
    }

    T front() const {
        if (isEmpty()) throw std::underflow_error("Deque is empty");
        return head->data;
    }

    T back() const {
        if (isEmpty()) throw std::underflow_error("Deque is empty");
        return tail->data;
    }

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }
};

bool isPalindrome(const std::string& str) {
    Deque<char> dq
    for (char ch : str) {
        dq.push_back(ch);
    }
    while (dq.size() > 1) {
        if (dq.pop_front() != dq.pop_back()) return false;
    }
    return true;
}

int main() {
    std::cout << "radar: " << (isPalindrome("radar") ? "YES" : "NO") <<
std::endl,
    std::cout << "hello: " << (isPalindrome("hello") ? "YES" : "NO") <<
std::endl,
    return 0;
}

```

Пример 4. Приоритетная очередь

Задача: Реализовать приоритетную очередь на основе кучи (max-heap).

```

#include <iostream>
#include <stdexcept>

```

```

template <typename T>
class PriorityQueue {
private:
    T* heap;
    int count;
    int capacity;

    void resize() {
        int newCapacity = capacity * 2;
        T* newHeap = new T[newCapacity];
        for (int i = 0; i < count; i++) newHeap[i] = heap[i];
        delete[] heap;
        heap = newHeap;
        capacity = newCapacity;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {
                std::swap(heap[index], heap[parent]);
                index = parent;
            } else break;
        }
    }

    void heapifyDown(int index) {
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;
            if (left < count && heap[left] > heap[largest]) largest = left;
            if (right < count && heap[right] > heap[largest]) largest = right;
            if (largest == index) break;
            std::swap(heap[index], heap[largest]);
            index = largest;
        }
    }

public:

```

```

PriorityQueue(int initialCapacity = 10) : capacity(initialCapacity),
count(0) {
    heap = new T[capacity];
}

~PriorityQueue() { delete[] heap; }

void push(const T& value) {
    if (count >= capacity) resize();
    heap[count] = value;
    heapifyUp(count);
    count++;
}

T pop() {
    if (isEmpty()) throw std::underflow_error("PriorityQueue is empty");
    T maxValue = heap[0];
    heap[0] = heap[count - 1];
    count--;
    heapifyDown(0);
    return maxValue;
}

T top() const {
    if (isEmpty()) throw std::underflow_error("PriorityQueue is empty");
    return heap[0];
}

bool isEmpty() const { return count == 0; }
int size() const { return count; }
};

int main() {
    PriorityQueue<int> pq;
    pq.push(30);
    pq.push(10);
    pq.push(50);
    pq.push(20);

    while (!pq.isEmpty()) {
        std::cout << pq.pop() << " ";
    }
}

```

```

std::cout << std::endl;
return 0;
}

```

Индивидуальные задания

Каждый вариант содержит 4 задания - по одному на каждую структуру данных: стек, очередь, дек, приоритетная очередь.

Вариант 1

Структура	Задание
Стек	Проверить правильность расстановки скобок в выражении (вида ([[]{}]))
Очередь	Смоделировать очередь в банке (время обслуживания клиентов)
Дек	Проверить, является ли строка палиндромом (с использованием дека)
Приоритетная очередь	Найти K наибольших элементов в массиве

Вариант 2

Структура	Задание
Стек	Вычислить значение выражения в обратной польской записи (например, 3 4 + 5 *)
Очередь	Реализовать очередь на двух стеках
Дек	Реализовать дек с фиксированной ёмкостью (кольцевой буфер)
Приоритетная очередь	Найти K наименьших элементов в массиве

Вариант 3

Структура	Задание
Стек	Перевести число из десятичной системы в двоичную

Структура	Задание
Очередь	Найти максимальный элемент в очереди за $O(1)$
Дек	Найти максимальный элемент в деке за $O(1)$
Приоритетная очередь	Сортировка с помощью приоритетной очереди (Heap Sort)

Вариант 4

Структура	Задание
Стек	Найти минимальный элемент в стеке за $O(1)$
Очередь	Найти минимальный элемент в очереди за $O(1)$
Дек	Найти минимальный элемент в деке за $O(1)$
Приоритетная очередь	Объединить K отсортированных списков

Вариант 5

Структура	Задание
Стек	Проверить, является ли строка палиндромом (с использованием стека)
Очередь	Реализовать циклическую очередь фиксированного размера
Дек	Реализовать дек с операцией <code>rotateLeft(k)</code>
Приоритетная очередь	Найти медиану потока чисел (две кучи)

Вариант 6

Структура	Задание
Стек	Сортировать стек (разрешается использовать дополнительный стек)

Структура	Задание
Очередь	Реализовать очередь с приоритетом (обычная + приоритетная)
Дек	Реализовать дек с операцией reverse() (разворот)
Приоритетная очередь	Реализовать приоритетную очередь с изменением приоритета

Вариант 7

Структура	Задание
Стек	Удалить все чётные элементы из стека
Очередь	Сортировать очередь (разрешается использовать дополнительную очередь)
Дек	Удалить все чётные элементы из дека
Приоритетная очередь	Симуляция планировщика задач (процессы с разными приоритетами)

Вариант 8

Структура	Задание
Стек	Удалить все нечётные элементы из стека
Очередь	Проверить, является ли очередь палиндромом
Дек	Удалить все нечётные элементы из дека
Приоритетная очередь	Реализовать min-heap вместо max-heap

Вариант 9

Структура	Задание
Стек	Проверить правильность HTML-разметки (например, <code><div><p></p></div></code>)
Очередь	Удалить все чётные элементы из очереди
Дек	Проверить, является ли дек палиндромом
Приоритетная очередь	Реализовать приоритетную очередь на основе бинарной кучи из массива

Вариант 10

Структура	Задание
Стек	Вычислить значение выражения с унарным минусом ($-5 + 3$)
Очередь	Удалить все нечётные элементы из очереди
Дек	Скольльзящее окно: найти максимумы во всех подмассивах длины k
Приоритетная очередь	Реализовать операцию merge (слияние двух приоритетных очередей)

Вариант 11

Структура	Задание
Стек	Преобразовать инфиксную запись в постфиксную (алгоритм сортировочной станции)
Очередь	Объединить две очереди в одну (чередую элементы)
Дек	Скольльзящее окно: найти минимумы во всех подмассивах длины k

Структура	Задание
Приоритетная очередь	Реализовать приоритетную очередь с фиксированной ёмкостью

Вариант 12

Структура	Задание
Стек	Реализовать стек с поддержкой операции <code>getMiddle()</code> (середина стека)
Очередь	Разделить очередь на две: чётные и нечётные позиции
Дек	Реализовать дек с операцией <code>insertAt(index)</code>
Приоритетная очередь	Проверить, является ли массив двоичной кучей

Вариант 13

Структура	Задание
Стек	Реализовать стек с поддержкой операции <code>reverse()</code> (разворот)
Очередь	Реализовать очередь с операцией <code>rotate(k)</code> (циклический сдвиг)
Дек	Реализовать дек с операцией <code>removeAt(index)</code>
Приоритетная очередь	Восстановить свойство кучи (<code>heapify</code>) для произвольного массива

Вариант 14

Структура	Задание
Стек	Реализовать два стека в одном массиве (экономное использование памяти)

Структура	Задание
Очередь	Реализовать очередь с операцией reverse() (разворот)
Дек	Реализовать дек, который возвращает сумму всех элементов за $O(1)$
Приоритетная очередь	Реализовать приоритетную очередь для пользовательских типов (с компаратором)

Вариант 15

Структура	Задание
Стек	Реализовать стек с поддержкой операции getMin() за $O(1)$
Очередь	Реализовать очередь, которая возвращает среднее арифметическое за $O(1)$
Дек	Реализовать дек на основе кольцевого буфера
Приоритетная очередь	Реализовать приоритетную очередь с поддержкой удаления произвольного элемента

Контрольные вопросы

1. В чём заключается принцип работы стека (LIFO)?
2. В чём заключается принцип работы очереди (FIFO)?
3. Чем дек отличается от обычной очереди?
4. Что такое приоритетная очередь? Где она применяется?
5. Как реализовать стек на основе динамического массива?
6. Как реализовать очередь на основе кольцевого буфера?
7. Почему приоритетную очередь обычно реализуют на куче?
8. Какова сложность операций push и pop в стеке?
9. Какова сложность операций enqueue и dequeue в очереди?
10. Какова сложность операций в приоритетной очереди? Почему?
11. Как проверить правильность расстановки скобок с помощью стека?
12. Как реализовать очередь на двух стеках? Какова сложность?
13. Как реализовать дек на основе двусвязного списка?

- 14.Как реализовать дек на основе кольцевого буфера?
- 15.Как реализовать стек с поддержкой получения минимума за $O(1)$?

Требования к отчёту

Титульный лист
 Цель работы
 Теоретическая часть
 Постановка задачи (4 задания согласно варианту)
 Листинг программы (4 реализации с комментариями)
 Результаты работы (скриншоты вывода)
 Анализ сложности для каждой операции
 Ответы на контрольные вопросы
 Выводы
 Ссылка на Git-репозиторий

Критерии оценивания (единые для всех)

Максимальный балл: 100

№	Критерий	Баллы
1	Стек (Stack) - реализация класса и решение задачи	25
1.1	Класс стека реализован корректно (push, pop, top, isEmpty)	10
1.2	Задача со стеком решена верно	10
1.3	Обработка ошибок (попытка pop из пустого стека)	5
2	Очередь (Queue) - реализация класса и решение задачи	25
2.1	Класс очереди реализован корректно (enqueue, dequeue, front, isEmpty)	10
2.2	Задача с очередью решена верно	10

№	Критерий	Баллы
2.3	Обработка ошибок (попытка dequeue из пустой очереди)	5
3	Дек (Deque) - реализация класса и решение задачи	25
3.1	Класс дека реализован корректно (push_front, push_back, pop_front, pop_back)	10
3.2	Задача с деком решена верно	10
3.3	Обработка ошибок (попытка pop из пустого дека)	5
4	Приоритетная очередь (PriorityQueue) - реализация и решение задачи	25
4.1	Класс приоритетной очереди реализован корректно (push, pop, top, isEmpty)	10
4.2	Задача с приоритетной очередью решена верно	10
4.3	Корректная работа кучи (heapifyUp/heapifyDown)	5
Итого		100

Детализация по каждому критерию

1. Стек (25 баллов)

Подкритерий	Баллы
Корректная реализация push (с расширением массива при необходимости)	3
Корректная реализация pop	2
Корректная реализация top	2

Подкритерий	Баллы
Корректная реализация isEmpty и size	1
Корректная работа деструктора (освобождение памяти)	2
Задача решена верно (алгоритм работает корректно на всех тестах)	10
Обработка ошибок (выброс исключения при pop из пустого стека)	5
<i>Итого</i>	<i>*25*</i>

2. Очередь (25 баллов)

Подкритерий	Баллы
Корректная реализация enqueue (с кольцевым буфером или расширением)	3
Корректная реализация dequeue	2
Корректная реализация front	2
Корректная реализация isEmpty и size	1
Корректная работа деструктора (освобождение памяти)	2
Задача решена верно (алгоритм работает корректно на всех тестах)	10
Обработка ошибок (выброс исключения при dequeue из пустой очереди)	5
<i>Итого</i>	<i>*25*</i>

3. Дек (25 баллов)

Подкритерий	Баллы
Корректная реализация push_front	3
Корректная реализация push_back	3
Корректная реализация pop_front	2
Корректная реализация pop_back	2
Корректная реализация front и back	2
Корректная реализация isEmpty и size	1
Корректная работа деструктора (освобождение памяти)	2
Задача решена верно (алгоритм работает корректно на всех тестах)	10
<i>Итого</i>	<i>*25*</i>

4. Приоритетная очередь (25 баллов)

Подкритерий	Баллы
Корректная реализация push (heapifyUp)	3
Корректная реализация pop (heapifyDown)	3
Корректная реализация top	2
Корректная реализация isEmpty и size	1
Корректное управление памятью (расширение массива, деструктор)	2
Задача решена верно (алгоритм работает корректно на всех тестах)	10
Корректная обработка пустой очереди	2
Правильная сложность $O(\log n)$ для push/pop	2

Подкритерий	Баллы
<i>Итого</i>	*25*

Штрафные баллы (вычитаются из общей суммы)

Нарушение	Штраф
Утечка памяти (не освобождена динамическая память в деструкторе)	-15
Двойное освобождение памяти	-15
Нет обработки попытки pop из пустой структуры	-10
Нет проверки выхода за границы в top()/front()	-5
Использование STL контейнеров вместо собственной реализации	-10
Код не отформатирован, нет комментариев	-5
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка (традиционная)	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60

Пример расчёта оценки

Студент выполнил:

Критерий	Максимум	Получено
Стек	25	22 (ошибка в расширении массива)
Очередь	25	20 (неверная обработка краевых случаев)
Дек	25	25 (всё верно)
Приоритетная очередь	25	18 (ошибка в heapify)
Штраф	-	-10 (утечка памяти в стеке)
Итого	100	75

Оценка: Хорошо (75 баллов)